

1. Introduction

This report presents the detailed implementation of a trace-driven multi-core cache coherence simulator developed for the CS4223 Mini-Project. The simulator supports three different cache coherence protocols:

- **MESI** – invalidation-based snooping protocol
- **Dragon** – update-based snooping protocol with no Invalid state
- **MOESI** – invalidation-based protocol with an additional Owned state

The project was completed in three stages:

- **Part 1** – Single-core CPU with cache, DRAM, and timing
- **Part 2** – Multi-core snooping bus with MESI coherence
- **Part 3** – Multi-core Dragon protocol
- **Advanced task** – Multi-core MOESI protocol as an optimisation

The simulator consumes PARSEC benchmark traces (blackscholes, bodytrack, fluidanimate) for 4 cores. For each run, it reports:

- Overall and per-core execution cycles
- Compute, idle, and memory cycles per core
- Load/store counts and cache hit/miss counts
- Bus data traffic (in bytes)
- Number of invalidations/updates
- Distribution of private vs shared data accesses

2. Development Environment

- **Language:** C++ (C++17)
- **Main libraries:**
 - I/O and containers: iostream, fstream, string, unordered_map, list, vector, queue, optional
 - Utilities: iomanip, cassert
- **C++ features used:**
 - std::optional for optional bus transactions
 - Range-based for loops
 - RAII for file streams and class lifetimes
- **Platform:** Portable; tested on Linux and Windows (GCC/Clang/MSVC).

Each protocol is implemented in its own translation unit:

- CacheSimulator_MESI.cpp
- CacheSimulator_DRAGON.cpp
- CacheSimulator_MOESI.cpp

All three share the same high-level architecture and differ mostly in:

- The cache line state enums

- The snoop() function
- How Bus::tick() interprets snoop results and sets timings/state transitions

3. Design Assumptions and Configuration

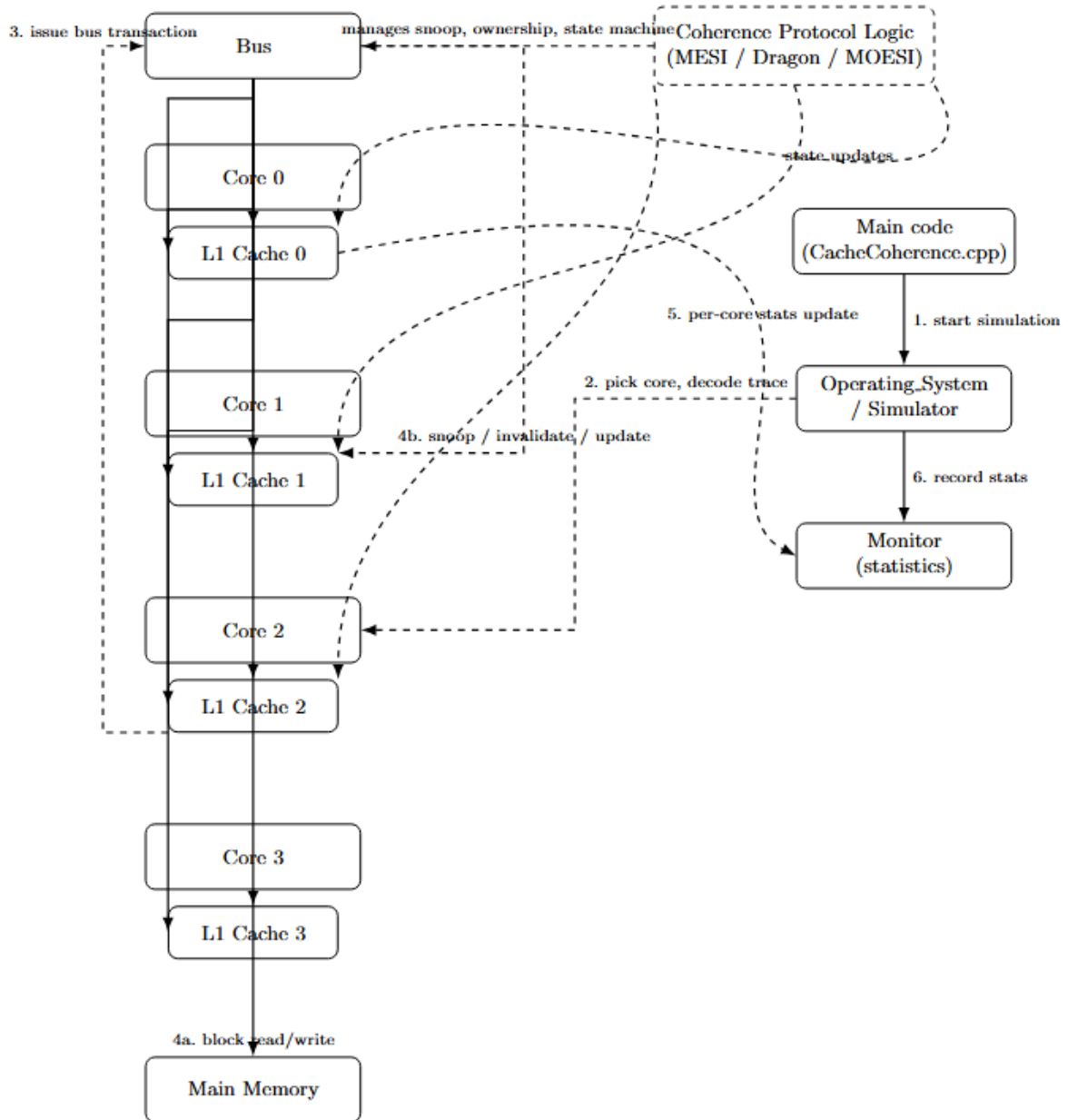


Figure 1: High-level design of the cycle-accurate multi-core cache coherence simulator

The simulator follows the assignment's assumptions and makes a few explicit design choices:

- **Address & data model**
 - 32-bit physical addresses
 - Word size: 4 bytes
 - Each memory reference accesses exactly one word

- **Cache hierarchy**
 - Each core has a private L1 data cache; no L2
 - Default configuration (if not overridden by CLI):
 - 4 KB, 2-way set associative
 - Block size: 32 bytes
 - Write-back, write-allocate
 - LRU replacement at the block granularity
- **Coherence and bus**
 - Snooping bus, blocking caches
 - On a miss, the core stalls until the cache finishes the transaction, but the cache still processes snoops
 - Bus arbitration: FIFO, first-come-first-serve, ties resolved arbitrarily
 - One outstanding bus transaction at a time (single shared bus)
- **Timing**
 - Cache hit: 1 cycle
 - Memory access: +100 cycles to fetch/write a block
 - Cache-to-cache transfer: 2 cycles per word (so $2 \times \text{block_size bytes} / \text{word_size}$ cycles)
 - Bus update of a single word (Dragon): 2 cycles, 4 bytes
 - Dirty eviction: 100 cycles to write a full block back to memory
- **Trace interpretation**
 - Each trace line is (type, value):
 - type = 0: load from word address value
 - type = 1: store to word address value
 - type = 2: compute; value = number of compute cycles between memory operations
 - Compute time is modeled as “gaps” in the core’s cycle counter between memory operations.

Additional assumptions (such as modelling bus upgrade latency as 1 cycle, not modelling address/control bus bandwidth, and treating some writeback timings slightly conservatively) are clearly encoded in the bus timing logic and kept consistent across protocols.

4. System Architecture Overview

The simulator uses an object-oriented design with clear separation of concerns:

- **Operating_System** – top-level orchestrator and simulation driver
- **Core** – per-core execution unit that consumes a trace file
- **LRU_Cache** – private L1 data cache per core, with coherence state
- **Bus** – shared snooping bus with FCFS arbitration
- **Monitor** – statistics collector and printer

Conceptually:

- Operating_System owns a global cycle counter, a Bus, a Monitor, and a vector of Core*.
- Each Core owns an LRU_Cache and a trace file.

- Each LRU_Cache implements:
 - Set-associative layout
 - Protocol-specific coherence state per line
 - LRU replacement and eviction writebacks
- The Bus:
 - Serializes transactions
 - Calls snoop() on all other caches
 - Decides timing and data source (memory vs other cache)
- Monitor:
 - Tracks and prints all metrics required by the assignment.

5. Common Components

5.1 Monitor

Monitor centralizes all statistics. Per core, it tracks:

- compute_cyc[i] – total cycles spent in compute (type-2) instructions
- num_loads[i], num_stores[i]
- idle_cyc[i] – cycles where core is stalled on a miss
- execution_cyc[i] – cycle at which the core finishes its trace
- hit_miss_cnt[i] – (hits, misses) for data cache

Global counters:

- overall_cyc – max_i execution_cyc[i]
- bus_data_traffic – sum of all data bytes moved over the bus
- bus_invalidate_update_cnt – number of invalidation (MESI/MOESI) or update (Dragon) events
- private_data_accesses – accesses to states considered private (e.g. M/E)
- shared_data_accesses – accesses to shared states (S, O, Sc, Sm)

The print_statistics() method maps these fields directly to items 1–8 in the assignment:

1. Overall and per-core execution cycles
2. Compute cycles per core
3. Number of loads/stores per core
4. Idle cycles per core
5. Cache hits and misses per core
6. Bus data traffic in bytes
7. Number of invalidations/updates
8. Private vs shared access counts

5.2 Operating_System

The Operating_System builds the whole system and runs the cycle-accurate simulation.

Construction

- Allocates a single shared global_cycle (pointer to int).

- Creates one Monitor.
- Creates one Bus, giving it the `global_cycle` and Monitor.
- Creates `n_cores` Core objects, passing each:
 - its core ID,
 - the benchmark base name,
 - pointers to `global_cycle`, Monitor, and Bus.

Main simulation loop

The simulator advances time one global cycle at a time. In each cycle, all components tick in a fixed order:

1. **Caches tick:** each L1 cache completes any pending 1-cycle hit and, if necessary, notifies its core that a memory operation has finished.
2. **Bus ticks:** the shared bus progresses the currently active coherence transaction or starts a new one from its queue, triggering snoops in other caches.
3. **Cores execute:** each core attempts to execute the next trace instruction (load, store, or compute), unless it is stalled waiting for a miss to complete.

After these three steps, the global cycle counter is incremented. The simulation stops when all cores have consumed their trace files and completed any outstanding memory operations. The Monitor then derives:

- per-core completion cycles,
- the overall execution cycle (maximum over all cores),
- and prints all final statistics.

A compact pseudocode view:

```
while (not all cores finished) {
    for each cache: cache.tick()    // complete 1-cycle hits
    bus.tick()                      // progress or start transaction
    for each core: core.execute_next_instruction()
    global_cycle++
}

monitor.compute_overall_cycles()
monitor.print_statistics()
```

Important decisions

- **Tick order: caches → bus → cores**
Ensures hits initiated in the previous cycle complete before new memory operations are issued, and that bus/coherence effects are visible before cores fetch the next instruction.
- **Synchronous cores**
All cores share the same `global_cycle` and execute in lockstep, which simplifies reasoning about interleavings and coherence events.
- **Termination**
The simulation terminates when all cores report “no more trace entries” and have no outstanding cache misses; the final `overall_cyc` is the maximum completion cycle among all cores.

5.3 Core

Each Core encapsulates:

- A private `LRU_Cache*` cache
- An input ifstream `fin` to `benchmark_i.data`
- Pointers to Monitor, `global_cycle`, and Bus
- Flags for:
 - `waiting_io` – blocked on a cache miss / bus transaction
 - `waiting_cal` – cycle when current compute finishes
 - `pending_compute_cycles` – accumulated compute cycles not yet committed

Execution logic

On each cycle, `execute_next_instruction()`:

1. If `waiting_io` is true:
 - The core is stalled on a miss.
 - It increments `idle_cyc[id]` and returns.
2. If we are still in a compute burst (`*global_cycle < waiting_cal`):
 - No new trace entry is fetched; we just return (compute cycles are accumulated in `pending_compute_cycles`).
3. Otherwise, we fetch the next trace line:
 - **Type 0 (load):**
 - Add `pending_compute_cycles` to `compute_cyc[id]`.
 - Reset `pending_compute_cycles`.
 - Set `waiting_io = true`.
 - Call `cache->get(address)`.
 - Increment `num_loads[id]`.
 - **Type 1 (store):**
 - Same pattern, but call `cache->put(address, data)` and increment `num_stores[id]`.
 - **Type 2 (compute):**
 - Interpret value as `cal_cycles`.
 - Set `waiting_cal = *global_cycle + cal_cycles`.
 - Accumulate `pending_compute_cycles += cal_cycles`.

When the cache finishes a hit/miss (via `finish_io()`), it clears `waiting_io` and the core can proceed.

This implements a blocking cache model where compute and memory are serialized per core, while still allowing snoop processing in the cache.

5.4 LRU Cache

LRU_Cache implements a set-associative cache with coherence state.

Organisation

- `cache_sets`: `vector<list<Entry>>` – each set is a doubly-linked list ordered by recency (front = MRU, back = LRU).
- `entry_location`: `unordered_map<int, list<Entry>::iterator>` mapping `block_memory` → iterator in the set list.
- Entry holds:
 - `address` – block-aligned address
 - `words` – vector of words
 - `state` – protocol-specific state: MESI / Dragon / MOESI enum

Address decoding

```
int block_memory = address / block_size;
```

```
int index      = block_memory % number_sets;
```

```
int offset     = (address % block_size) / word_size;
```

LRU operations

- On every hit, `reorder(index, block_memory)` moves the node to the front of the list in $O(1)$ using `list::splice`.
- On miss when a set is full, we evict the entry at the back (LRU):
 - If its state is dirty (M / Sm / O), we issue a FlushWB request to DRAM via the bus.
 - Otherwise we just drop it.

Hit timing

- On a hit, `get()/put()`:
 - Records statistics
 - Sets `pending_hit_completion` = true
 - Sets `hit_complete_at_cycle` = `*global_cycle + 1`
- `tick()` checks if the hit completion time has been reached:
 - When it is, it clears the flag and calls `core->finish_io()`.

Miss handling and request queue

- On a miss, the cache:

- Creates or reuses an Entry,
- Enqueues a Request with a bus command (BusRd, BusRdX, BusUpgr, BusUpd, or FlushWB),
- Increments `n_waiting_io`,
- If this is the only outstanding request, immediately forwards it to the bus.

When the bus indicates completion (`notify_finish_io()`):

- The cache decrements `n_waiting_io`.
- If there are more queued Requests for this core, it forwards the next one to the bus.
- If `n_waiting_io` reaches 0, it calls `core->finish_io()`.

This mechanism allows a core to have multiple queued actions (e.g. Dragon write miss: BusRd followed by conditional BusUpd) but still appear blocking from the core's perspective.

5.5 Bus

The Bus models the shared snooping bus and enforces FCFS arbitration.

State

- pending: `queue<BusTxn>` – transactions waiting to be serviced.
- active: `optional<BusTxn>` – current transaction.
- Pointers to:
 - `global_cycle`
 - Monitor
 - `Operating_System` (to iterate over cores and caches)

BusTxn

A BusTxn contains:

- `cmd` – command (BusRd, BusRdX, BusUpgr, BusUpd, FlushWB)
- `address` – block or word address
- `src_core` – requesting core
- `remaining_cycles`
- `supplied_by_cache` – 0 = memory, 1 = clean cache, 2 = dirty cache
- `supplier_core` – which cache supplied (if any)
- `bytes_on_bus`
- `num_invalidations` (MESI/MOESI)
- `word_value` (for BusUpd)

Tick behaviour

On each cycle:

1. Finish current transaction (if any)
 - If active exists and its `remaining_cycles == 1`, we:
 - Add `bytes_on_bus` to `bus_data_traffic`.

- Increment bus_invalidate_update_cnt for invalidation or update commands.
 - Call notify_finish_io() on the requester's cache.
 - Clear active.
- 2. Start next transaction (if any)
 - If active is empty and pending is non-empty:
 - Pop front as new active.
 - Call snoop(cmd, address, active) on all other caches.
 - From snooping we learn:
 - Whether data is supplied by a cache and whether it is dirty/clean.
 - How many caches invalidated their copies (BusRdX / BusUpgr).
 - For Dragon, how many sharers remain (for BusUpd).
 - Based on this, we set:
 - bytes_on_bus and remaining_cycles
 - New coherence state of the requester's line (via update_entry_state style logic in each protocol's implementation).

This bus behaviour is reused across MESI, Dragon, and MOESI. The differences lie in:

- snoop responses in each protocol's LRU_Cache::snoop()
- how supplied_by_cache is interpreted
- how states change when a transaction finishes.

6. Single-Core Cache Simulator

Part 1 is essentially a restricted version of the above architecture with one core and no coherence states:

- The L1 cache is still set-associative with LRU and write-back.
- There is no Bus or snooping; the cache talks directly to DRAM.
- The per-block metadata reduces to {tag, dirty}.

Load/store handling:

- On a hit:
 - Add 1 cycle for cache access.
- On a miss:
 - Add 1 cycle for cache lookup + 100 cycles to read from DRAM.
 - If an evicted block is dirty, add 100 cycles to write it back.
- Monitor still tracks total cycles, hits, misses, dirty writebacks, and bus bytes (now interpreted as DRAM transfers).

Part 1 served primarily to:

- Validate the basic LRU and timing logic,
- Provide a reference for cache hit/miss statistics before introducing coherence.

7. MESI Protocol

MESI is the baseline invalidation-based protocol.

7.1 State and commands

- States: M, E, S, I
- Commands: BusRd, BusRdX, BusUpgr, FlushWB

7.2 Processor-side operations

Loads (get)

- Hit in M or E:
 - Count as private access, 1-cycle hit.
- Hit in S:
 - Count as shared access, 1-cycle hit.
- Miss or I:
 - Count miss, possibly evict LRU (writeback if M).
 - Issue BusRd.
 - Final state:
 - E if no other cache supplied data
 - S if snooping detected sharers

Stores (put)

- Hit in M:
 - Private 1-cycle write, state remains M.
- Hit in E:
 - Silent E $\rightarrow$ M upgrade, 1-cycle write, no bus traffic.
- Hit in S:
 - Shared write: issue BusUpgr to invalidate other copies, then transition to M.
- Miss or I:
 - Write miss: issue BusRdX, install line in M.

7.3 Snooping rules

On other cores' transactions:

- BusRd:
 - If we hold line in M: supply dirty data, M $\rightarrow$ S.
 - If we hold line in E: supply clean data, E $\rightarrow$ S.
 - S stays S.
- BusRdX:
 - If M: supply dirty data, M $\rightarrow$ I.
 - If E or S: $\rightarrow$ I (invalidate).
- BusUpgr:
 - If S: S $\rightarrow$ I.

These rules ensure at most one M/E copy and any number of S copies; writes invalidate all sharers.

7.4 Bus timing for MESI

- BusRd:
 - Dirty supplier (M): cache-to-cache transfer + flush → more cycles and bytes.
 - Clean supplier (E): cache-to-cache transfer only.
 - No supplier: fetch from memory (100 cycles).
- BusRdX:
 - Similar to BusRd, but requestor always ends in M.
- BusUpgr:
 - Invalidation only; no data transfer.

8. Dragon Protocol

Dragon is an update-based protocol with **E**, **Sc**, **Sm**, **M** states and no Invalid state.

8.1 State and commands

- States:
 - E – exclusive, clean
 - Sc – shared, clean, not owner
 - Sm – shared, modified, owner (responsible for writeback)
 - M – modified, private
- Commands:
 - BusRd, BusUpd, FlushWB

8.2 Processor-side operations

Loads (get)

- Hit in M or E: private access, 1 cycle.
- Hit in Sc or Sm: shared access, 1 cycle.
- Miss:
 - Evict LRU if needed (writeback if dirty).
 - Insert line as E.
 - Issue BusRd, state may later become Sc if sharers exist.

Stores (put)

- Hit in M: private write, stay M.
- Hit in E: silent E → M upgrade.
- Hit in Sc:
 - Shared write: issue BusUpd to update all sharers, final state becomes Sm if others still have copies or M otherwise.
- Hit in Sm:
 - Owner write: issue BusUpd to update sharers, stay Sm.
- Miss:

- Evict if needed.
- Insert line as E with updated word.
- Enqueue BusRd and a *potential* BusUpd.
- After BusRd:
 - If no sharers, $E \rightarrow M$ and BusUpd is cancelled.
 - If sharers, $E \rightarrow Sc$ and BusUpd executes, leading to $Sc \rightarrow Sm$.

8.3 Snooping rules

- BusRd:
 - If E: supply clean data and downgrade $E \rightarrow Sc$.
 - If M: supply dirty data and convert to Sm.
 - If Sm: supply dirty data (stay Sm).
 - If Sc: do not supply but assert “shared” (S) signal.
- BusUpd:
 - Receivers with Sc: update word, stay Sc.
 - Receivers with Sm: update and downgrade $Sm \rightarrow Sc$ (new writer becomes owner).
 - Rare case: E also becomes Sc.

There is no invalidation; sharers keep valid copies and are updated on writes.

8.4 Bus timing for Dragon

- BusRd:
 - Cache-to-cache transfer if another cache holds the line (E/M/Sm).
 - Memory fetch otherwise.
- BusUpd:
 - 2 cycles, 4 bytes of data.
 - Requester’s final state:
 - Sm if there are other sharers.
 - M if it becomes the only holder (S’ case).

9. MOESI Protocol (Advanced task)

MOESI extends MESI with an Owned (O) state to reduce memory writebacks for dirty shared data.

9.1 State and commands

- States: M, O, E, S, I
- Commands: BusRd, BusRdX, BusUpgr, FlushWB

Key idea: O = shared, dirty, single owner responsible for supplying data and eventually writing back to memory.

9.2 Processor-side operations

Processor-side behaviour (get/put) is almost identical to MESI, with one difference:

- Accesses to O are treated like shared accesses (similar to S), but the cache knows it may have to supply data on snoops.

9.3 Snooping rules

- BusRd:
 - $M \rightarrow O$ and supply dirty data.
 - $O \rightarrow O$ and supply dirty data.
 - $E \rightarrow S$ and supply clean data.
 - S remains S and does not supply.
- BusRdX:
 - Any of M, O, E, S become I.
 - If M or O, also supply dirty data to the requester.
- BusUpgr:
 - S and O become I (invalidation).

Result:

- When a modified block becomes shared, it moves from M to O in the supplier and S in the requester, without immediately writing back to memory.

9.4 Bus timing for MOESI

- BusRd with dirty supplier (M or O):
 - Cache-to-cache transfer plus eventual writeback; modelled as:
 - $\text{bytes_on_bus} = 2 * \text{block_size} + \text{block_size}$
 - $\text{remaining_cycles} = 2 * \text{block_size} / \text{word_size} + 100$
 - Requester enters S, supplier enters or remains O.
- BusRd with clean supplier (E):
 - Same as MESI.
- BusRd from memory:
 - Same as MESI ($I \rightarrow E$).
- BusRdX / BusUpgr:
 - Same semantics as MESI, but starting from M/O/E/S.

This reduces DRAM traffic compared to MESI in situations where modified data is shared after being written.

10. Conclusion

Our report has described the design and implementation of a trace-driven multi-core cache coherence simulator implementing three protocols: MESI, Dragon, and MOESI.

Key design points

- A modular architecture separates common infrastructure (Operating_System, Core, LRU_Cache skeleton, Bus, Monitor) from protocol-specific coherence logic.
- The simulator is cycle-accurate at the level of cache hit/miss and bus/memory timing, using a single global clock and a strict tick order.

- The L1 caches use $O(1)$ LRU operations via hash maps plus linked lists and support write-back, write-allocate policies.

Protocol differences

- **MESI** is a standard invalidation-based protocol, efficient for private data but requiring invalidations and potential writebacks when data becomes shared.
- **Dragon** uses update-based coherence, avoiding invalidations and keeping sharers valid at the cost of more frequent small updates.
- **MOESI** optimizes MESI by introducing an Owned state, allowing dirty data to be shared without immediate writeback, reducing memory traffic.

Appendix

Running experience output, scripts and drawing scripts: [Running appendix](#).

High level implementation diagram code:

```
\documentclass{article}

\usepackage{tikz}
\usetikzlibrary{arrows.meta,positioning}

\begin{document}

\begin{figure}[h]
\centering
\begin{tikzpicture}[
  font=\small,
  block/.style={rectangle,draw,rounded corners,minimum width=3.3cm,minimum
height=1.0cm,align=center},
  smallblock/.style={rectangle,draw,rounded corners,minimum width=2.6cm,minimum
height=0.8cm,align=center},
  proto/.style={rectangle,draw,dashed,rounded corners,minimum width=4.3cm,minimum
height=1.2cm,align=center},
  line/.style={-Latex,thick},
  dashedline/.style={-Latex,thick,dashed}
]

% === Right side: main + simulator + monitor (now further right) ===
\node[block] (main) at (10,0) {Main code\\(CacheCoherence.cpp)};
\node[block,below=1.4cm of main] (sim) {Operating\_System\\ / Simulator};
\node[block,below=1.4cm of sim] (monitor) {Monitor\\(statistics)};

\draw[line] (main.south) -- node[right]{\scriptsize 1.\ start simulation} (sim.north);
\draw[line] (sim.south) -- node[right]{\scriptsize 6.\ record stats} (monitor.north);
```

```

% === Left: cores & caches ===
\node[block] (bus) at (0,3) {Bus};

\node[block,below=1.0cm of bus] (p0) {Core 0};
\node[smallblock,below=0.25cm of p0] (c0) {L1 Cache 0};

\node[block,below=1.4cm of c0] (p1) {Core 1};
\node[smallblock,below=0.25cm of p1] (c1) {L1 Cache 1};

\node[block,below=1.4cm of c1] (p2) {Core 2};
\node[smallblock,below=0.25cm of p2] (c2) {L1 Cache 2};

\node[block,below=1.4cm of c2] (p3) {Core 3};
\node[smallblock,below=0.25cm of p3] (c3) {L1 Cache 3};

\node[block,below=1.5cm of c3] (mem) {Main Memory};

% core <-> cache links
\foreach \p/\c in {p0/c0, p1/c1, p2/c2, p3/c3} {
    \draw[line] (\p.south) -- (\c.north);
}

% bus connections (wider spacing left→middle)
\draw[line] (bus.south) -- ++(0,-0.3) -| (c0.west);
\draw[line] (bus.south) -- ++(0,-2.0) -| (c1.west);
\draw[line] (bus.south) -- ++(0,-3.7) -| (c2.west);
\draw[line] (bus.south) -- ++(0,-5.4) -| (c3.west);

% memory
\draw[line] (bus.south) -- ++(0,-1.0) -- (mem.north)
node[above] {\scriptsize 4a.\ block read/write};

% === Coherence Protocol Logic (moved further right for more width) ===
\node[proto,right=4.5cm of bus] (proto)
    {Coherence Protocol Logic\\(MESI / Dragon / MOESI)};

% protocol → bus
\draw[dashedline] (proto.west) -- node[above]{\scriptsize manages snoop, ownership, state
machine} (bus.east);

% protocol → caches (positioned further right for spacing)
\draw[dashedline] (proto.south east) to[out=-60,in=90] node[right,pos=0.4]{\scriptsize state
updates} (c0.north east);
\draw[dashedline] (proto.south) to[out=-70,in=90] (c1.north east);
\draw[dashedline] (proto.south west)to[out=-80,in=90] (c2.north east);

% sim → cores

```

```

\draw[dashedline] (sim.west) -- node[above,sloped]{\scriptsize 2.\ pick core, decode trace}
++(-3.4,0)
|- (p2.east);

% cache → monitor
\draw[dashedline] (c0.east) .. controls +(10,2) and +(-2,1.0) ..
node[above]{\scriptsize 5.\ per-core stats update} (monitor.west);

% cache miss → bus
\draw[dashedline] (c2.south west) -- ++(-1.0,0) |-
node[above,sloped]{\scriptsize 3.\ issue bus transaction} (bus.west);

% bus → other caches
\draw[dashedline] (bus.east) -- ++(2.5,0) |-
node[above,sloped]{\scriptsize 4b.\ snoop / invalidate / update} (c1.north east);

% memory transfer highlight
% \draw[dashedline] (mem.east) -- ++(0,0) -- (c3.east)
% node[below]{\scriptsize 4a.\ block read/write};

\end{tikzpicture}
\caption{High-level design of the cycle-accurate multi-core cache coherence simulator.}
\end{figure}

\end{document}

```

AI Declaration

ChatGPT 5 is used to help us clarify the understanding of cache protocols, write profiling scripts and help us refine wording and grammar in our reports.